



3주차 발표

DL 3팀 강유진 김재희 박혜린

목차

5장 합성곱 신경망 I

#5.1 합성곱 신경망

#5.2 합성곱 신경망 맛보기



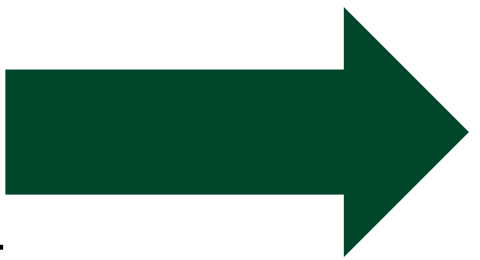
5.1 합성곱 신경망



#5.1.1 합성곱층의 필요성

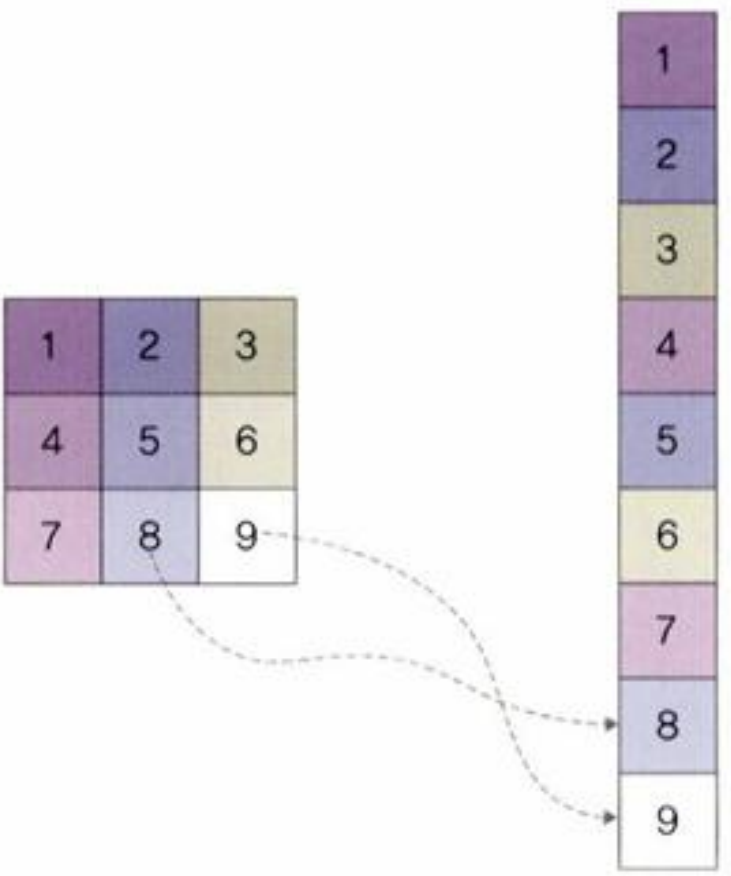
역전파 과정

- 복잡함
- 많은 자원 요구
- 시간 오래 걸림



합성곱 신경망

- 시간과 자원 절약
- 이미지의 세밀한 부분까지 분석



↑ 이미지 분석
방법

데이터의 공간적 구조를
무시하게 됨

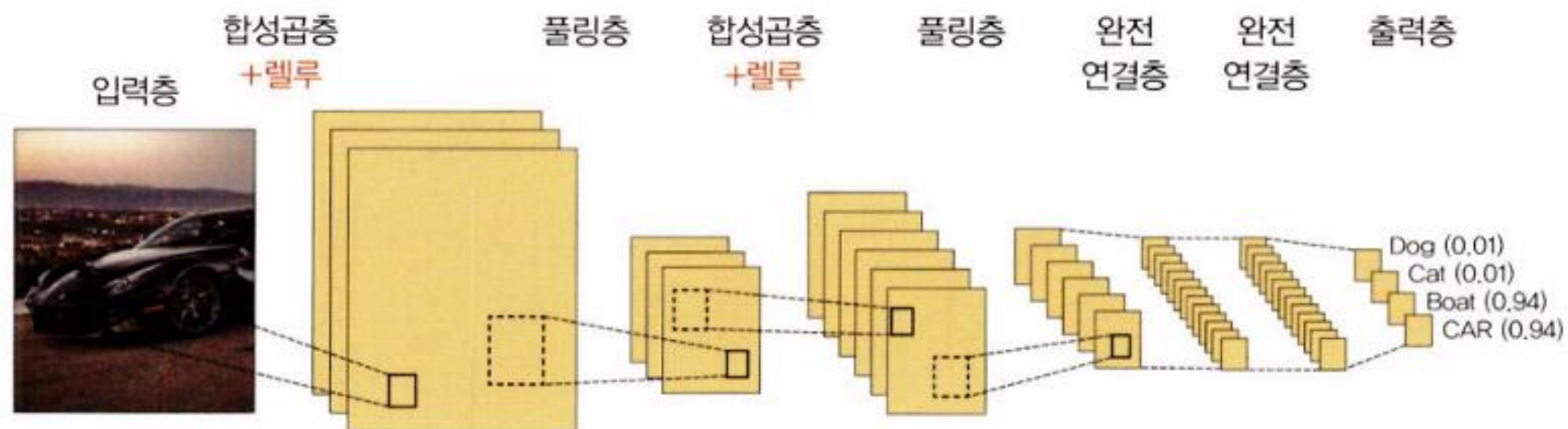


합성곱층

#5.1.2 합성곱 신경망 구조

합성곱 신경망: 음성 인식이나 이미지/영상 인식에서 주로 사용되는 신경망

1. 입력층
2. 합성곱층
3. 풀링층
4. 완전연결층
5. 출력층



#5.1.2 합성곱 신경망 구조

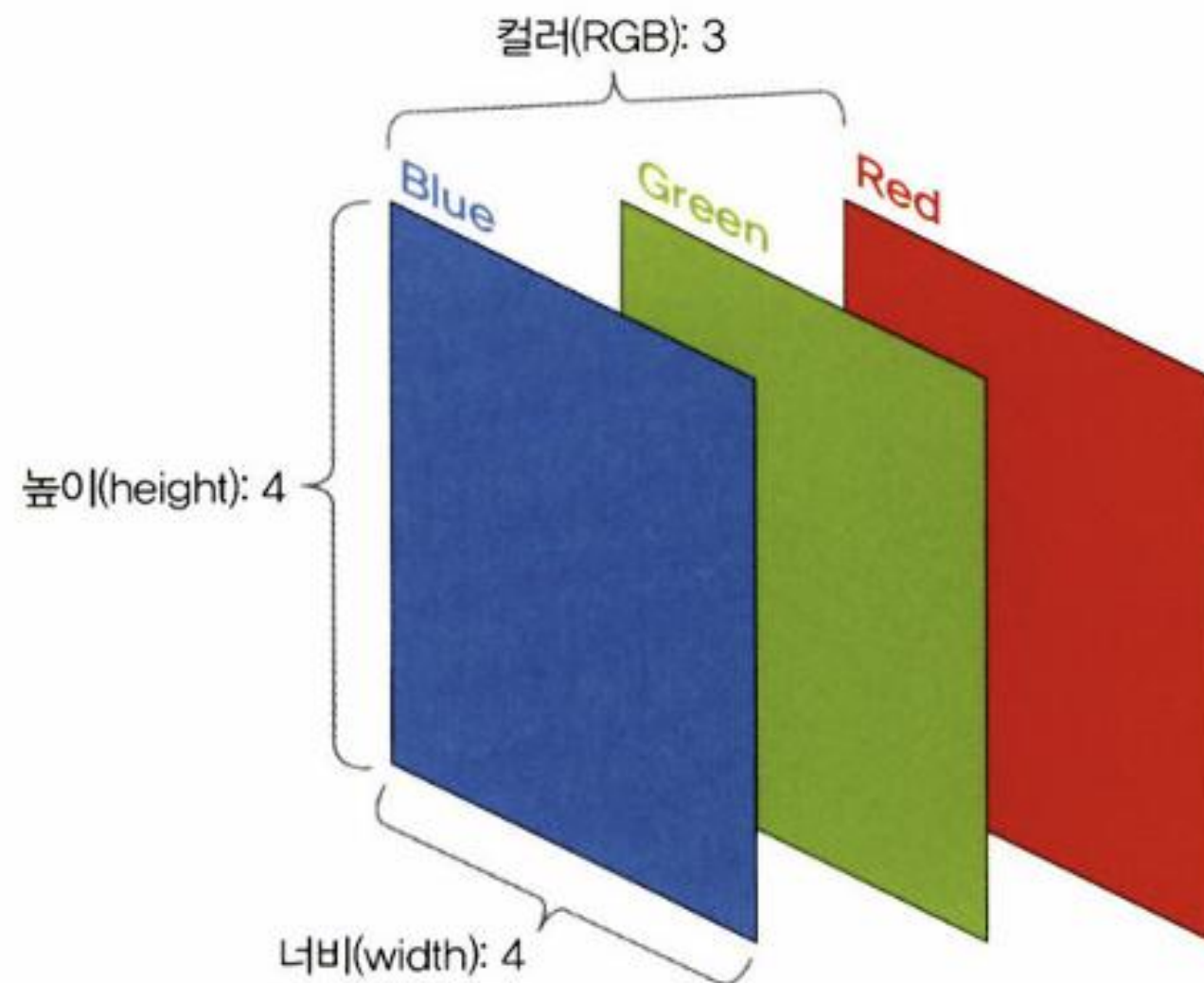
1. 입력층

입력 이미지 데이터가 최초로 거치게 되는 계층

채널:

이미지가 그레이스케일 → 1

이미지가 컬러 → 3



#5.1.2 합성곱 신경망 구조

2. 합성곱층

입력 데이터에서 특성을 추출하는 역할 수행

특성 추출을 할 때, 입력 이미지에 대한 특성을 감지하기 위해 커널이나 필터를 사용함
특성 맵: 커널 / 필터가 이미지의 모든 영역을 훑으며 특성을 추출한 결과물

커널은 3×3 , 5×5 크기로 적용되는 것이 일반적이며, 스트라이드에 따라 순차적으로 이동함

#5.1.2 합성곱 신경망 구조

2. 합성곱층

1단계. 입력 이미지에 3 x 3 필터 적용

입력 이미지와 필터를 포개고 대응되는 숫자끼리 곱한 후 모두 더함



#5.1.2 합성곱 신경망 구조

2. 합성곱층

2단계. 필터가 1만큼 이동



#5.1.2 합성곱 신경망 구조

2. 합성곱층

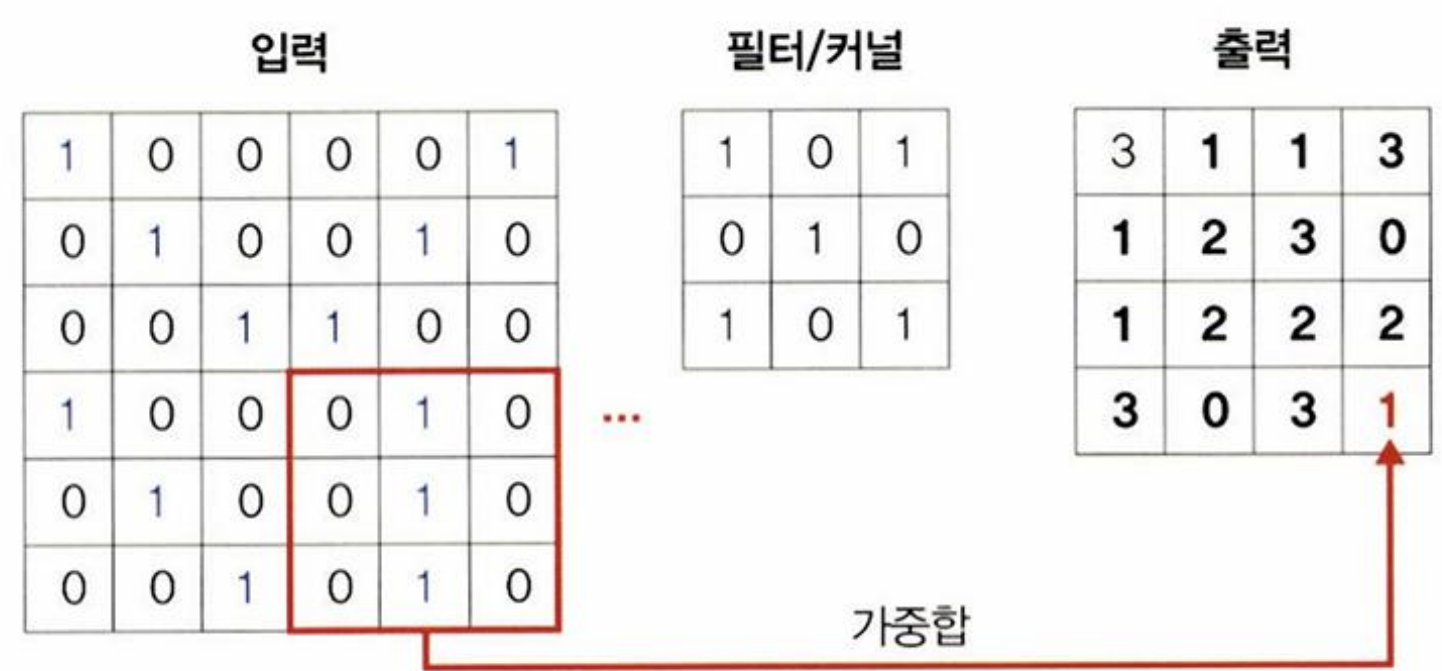
3, 4단계. 필터가 1만큼 다시 이동



#5.1.2 합성곱 신경망 구조

2. 합성곱층

5, 6단계. 필터가 1만큼 다시 이동

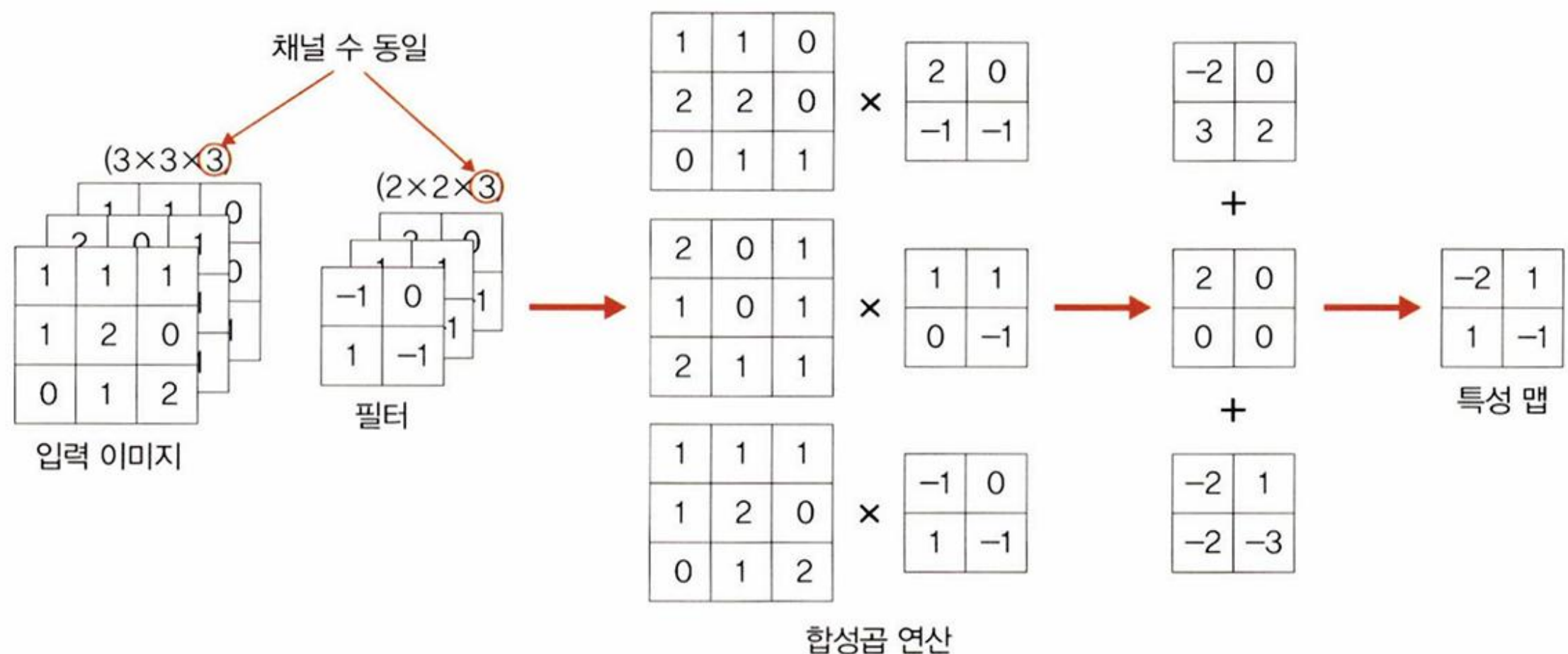


#5.1.2 합성곱 신경망 구조

2. 합성곱층

컬러 이미지의 합성곱

- 필터 채널이 3
- RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더함
- ★필터 개수 한 개

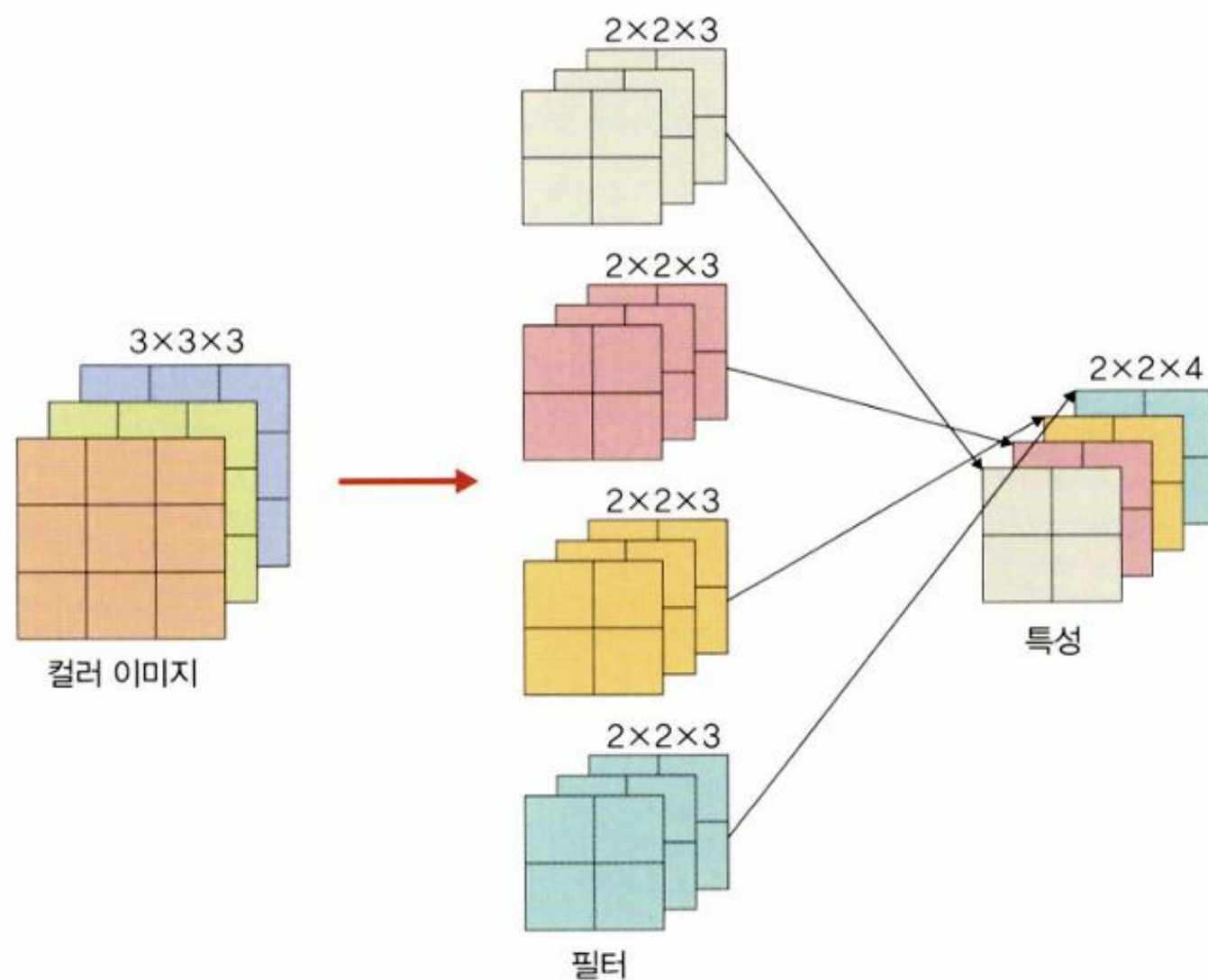


#5.1.2 합성곱 신경망 구조

2. 합성곱층

필터가 두 개 이상인 합성곱

필터 각각이 특성 추출 결과의 채널이 됨



- 입력 데이터: $W_1 \times H_1 \times D_1$ (W_1 : 가로, $\times H_1$: 세로, $\times D_1$: 채널 또는 깊이)
- 하이퍼파라미터
 - 필터 개수: K
 - 필터 크기: F
 - 스트라이드: S
 - 패딩: P
- 출력 데이터
 - $W_2 = (W_1 - F + 2P) / S + 1$
 - $H_2 = (H_1 - F + 2P) / S + 1$
 - $D_2 = K$

#5.1.2 합성곱 신경망 구조

3. 풀링층

특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고,
주요한 특성 벡터를 추출하여 학습을 효과적으로 할 수 있게 함



★ **최대 풀링**: 대상 영역에서 최대값을 추출

평균 풀링: 대상 영역에서 평균을 반환

#5.1.2 합성곱 신경망 구조

3. 풀링층

최대 풀링 과정

최댓값 선택



#5.1.2 합성곱 신경망 구조

3. 풀링층

평균 풀링 과정

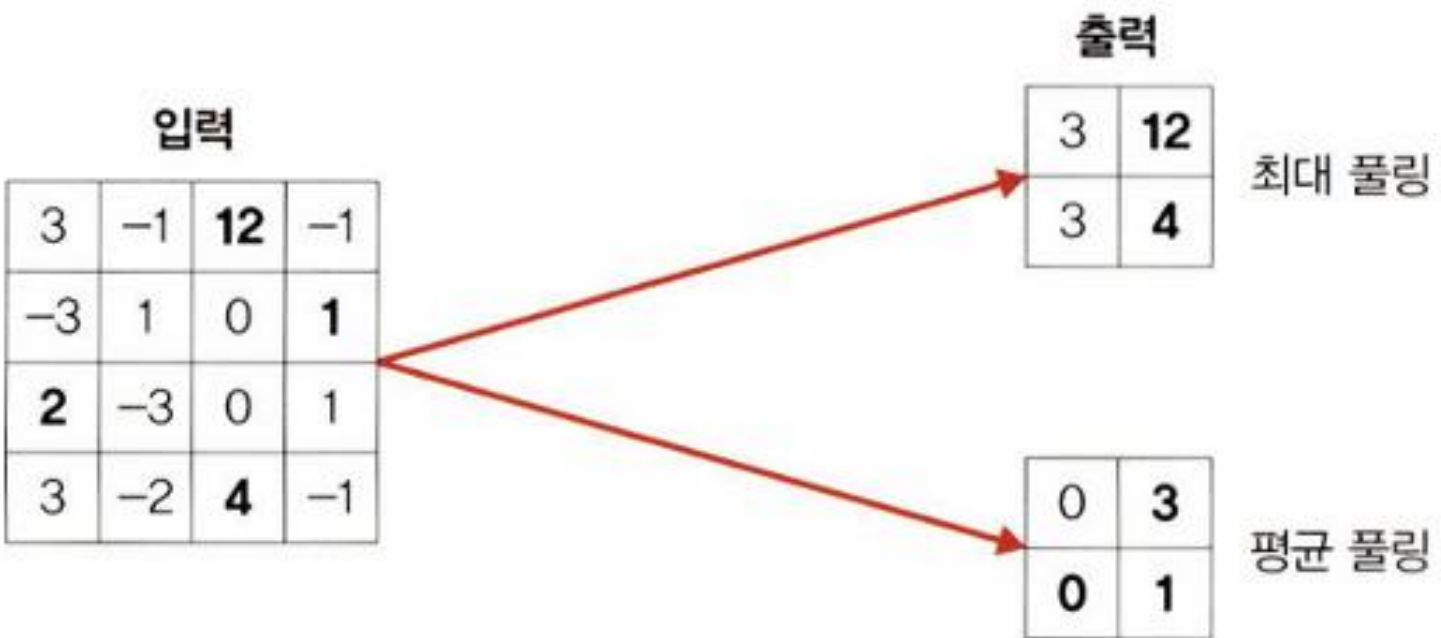
평균값 계산

$$0 = (3 + (-1) + (-3) + 1) / 4$$

$$3 = (12 + (-1) + 0 + 1) / 4$$

$$0 = (2 + (-3) + 3 + (-2)) / 4$$

$$1 = (0 + 1 + 4 + (-1)) / 4$$

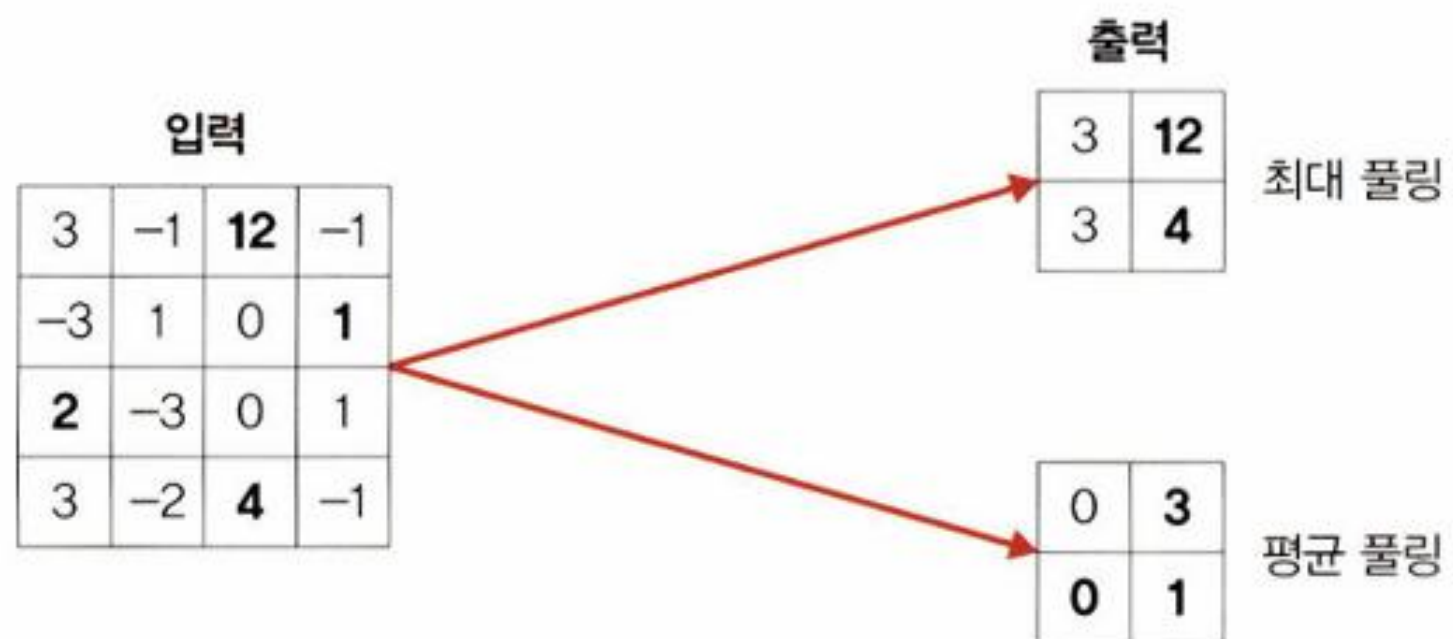


↑ 최대 풀링과 평균 풀링 비교

#5.1.2 합성곱 신경망 구조

3. 풀링층

최대 풀링과 평균 풀링 비교



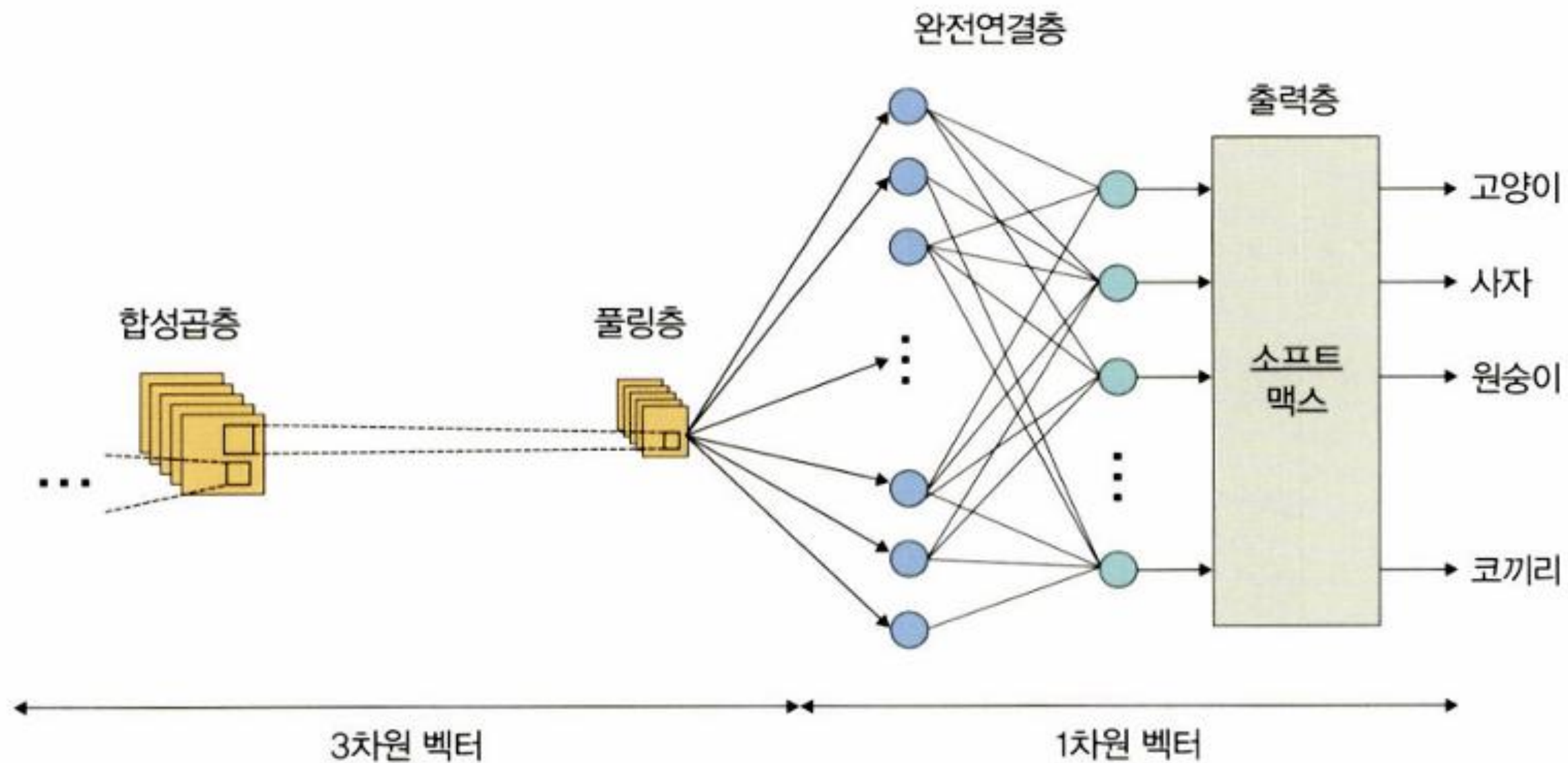
- 입력 데이터: $W_1 \times H_1 \times D_1$
- 하이퍼파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

↑ 최대 풀링과 평균 풀링 비교

#5.1.2 합성곱 신경망 구조

4. 완전연결층

이미지가 3차원 벡터에서 1차원 벡터로 펼쳐지게 됨



#5.1.2 합성곱 신경망 구조

5. 출력층

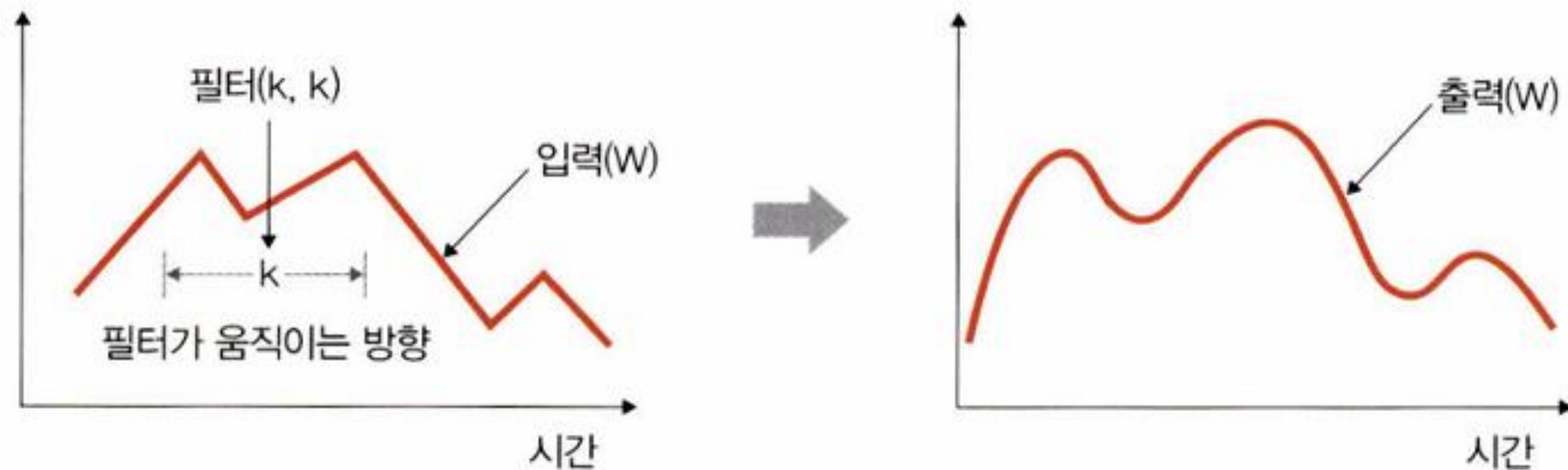
- 소프트맥스 활성화 함수가 사용됨
- 이미지가 각 레이블에 속할 확률 값이 출력되며, 이때 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정됨

#5.1.3 1D, 2D, 3D 합성곱

합성곱은 이동하는 방향의 수와 출력 형태에 따라 1D, 2D, 3D로 분류할 수 있음

1D 합성곱

- 필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱
- 입력(W)과 필터(k)에 대한 출력은 W가 됨 e.g. 입력 [1,1,1,1,1]이고 필터 [0.25,0.5,0.25]면 출력 [1,1,1]

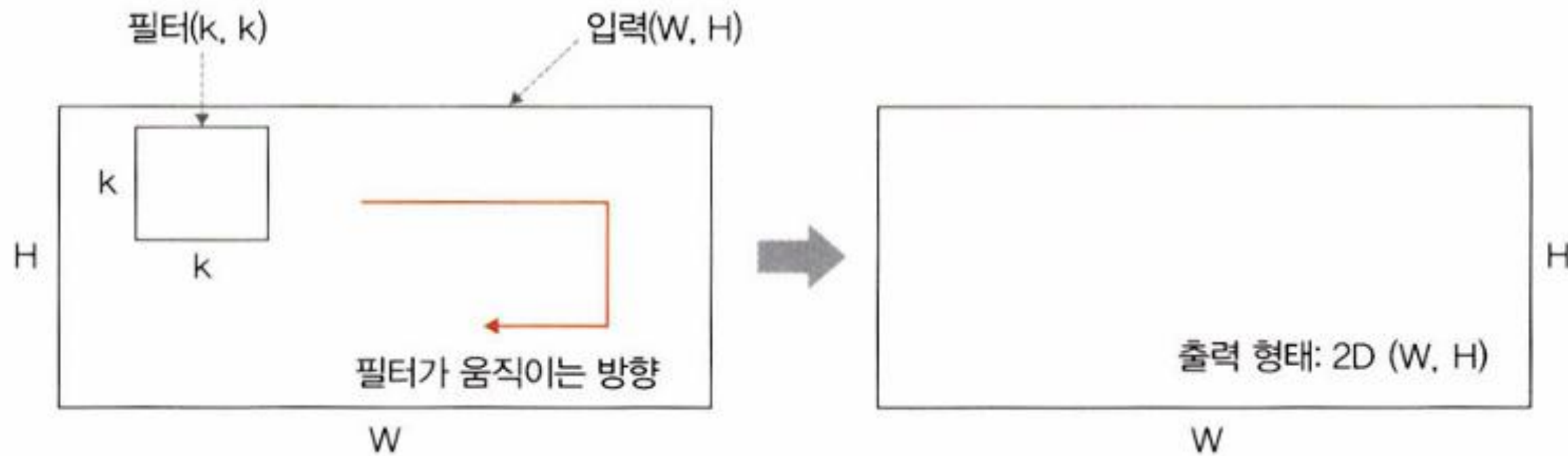


- 입력: W 너비(Width)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width)

#5.1.3 1D, 2D, 3D 합성곱

2D 합성곱

- 필터가 방향 두 개로 움직임
- 입력(W,H)과 필터(k,k)에 대한 출력은 (W,H)가 되며, 출력 형태는 2D 행렬이 됨

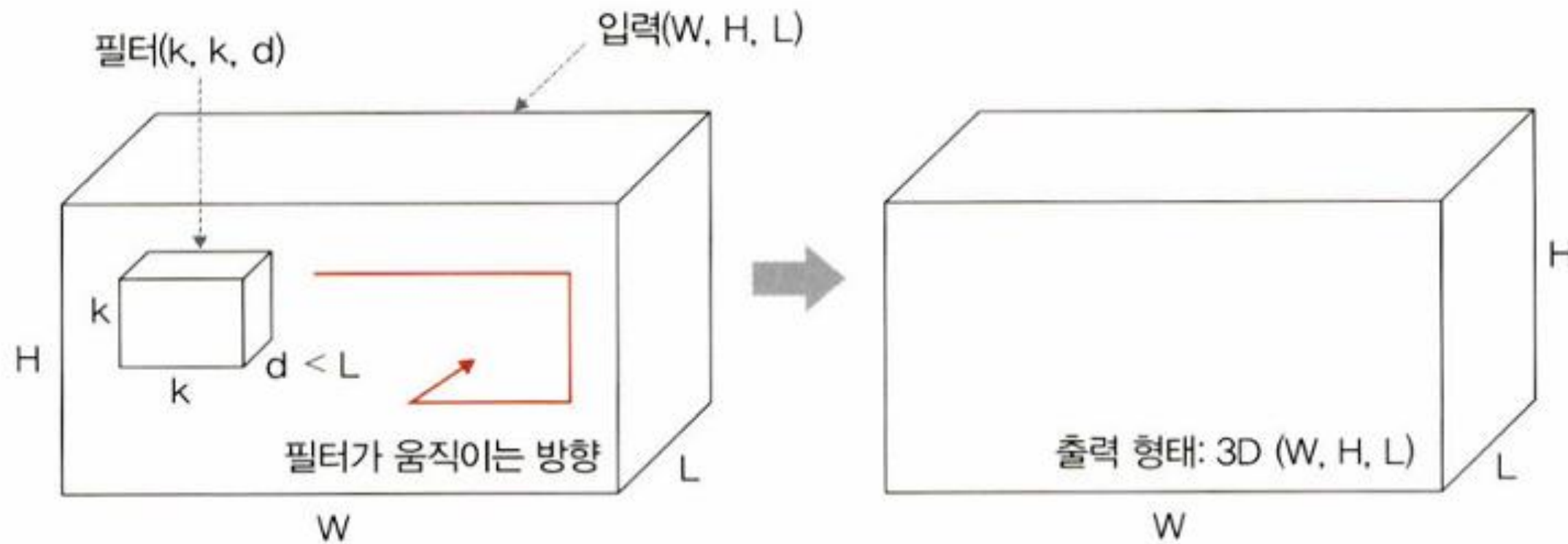


- 입력: W 너비(Width), H 높이(Height)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width), H 높이(Height)

#5.1.3 1D, 2D, 3D 합성곱

3D 합성곱

- 필터가 방향 두 개로 움직임
- 입력(W,H)과 필터(k,k)에 대한 출력은 (W,H)가 되며, 출력 형태는 2D 행렬이 됨

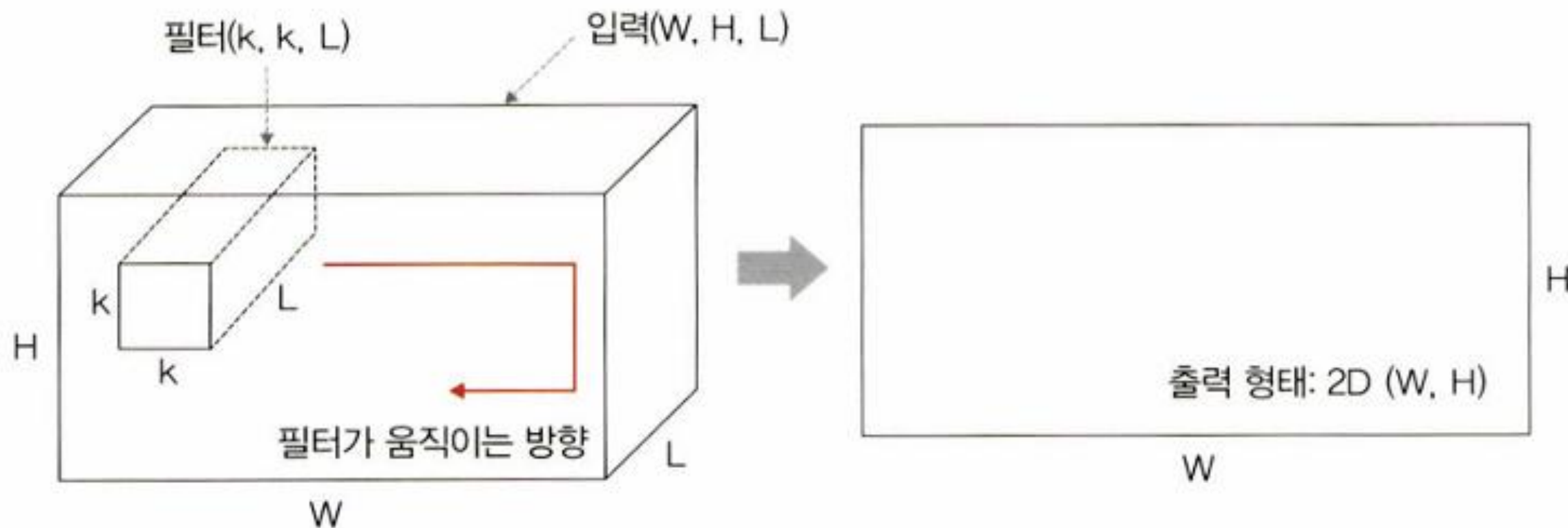


- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k$ (높이 $\times$ 너비), d: 깊이(depth)
- 출력: W 너비(Width), H 높이(Height), L 길이(Length)

#5.1.3 1D, 2D, 3D 합성곱

3D 입력을 갖는 2D 합성곱

- 입력이 3D 형태임에도 출력 형태가 3D가 아닌 2D 행렬을 취하는 경우
- 필터 길이가 입력 데이터 길이와 같을 때 만들어짐
- 필터는 두 방향으로만 움직이며 출력 형태는 2D

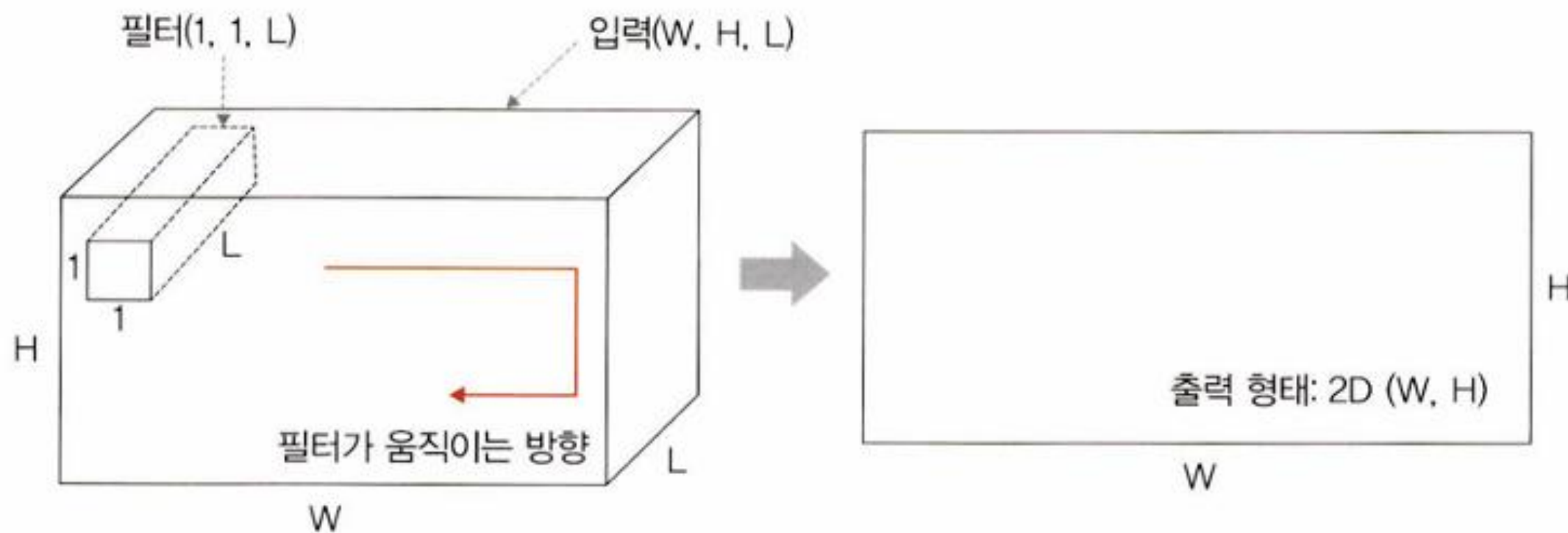


- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k$ (높이 $\times$ 너비), L 길이(Length)
- 출력: W 너비(Width), H 높이(Height)

#5.1.3 1D, 2D, 3D 합성곱

1x1 합성곱

- 3D 형태로 입력됨
- 채널 수를 조정해서 연산량을 줄이는 효과



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: 1×1(높이×너비), L 길이(Length)
- 출력: W 너비(Width), H 높이(Height)

5.2 합성곱 신경망 맛보기 (1)



#01 Fashion MNIST 데이터셋

#1 fashion_minst 데이터셋이란?

- Torchvision에 내장된 예제 데이터
- 작은 흑백 의류 이미지 모음
- MNIST 데이터셋처럼 10가지로 분류
- 28x28 픽셀 이미지 7만개

#2 데이터셋 구성요소

train_images: 0~255 값을 갖는 28x28 크기의
NumPy 배열

train_labels: 0~9 사이의 정수값을 갖는 배열

#3 레이블과 클래스



```
0 : T-Shirt
1 : Trouser
2 : Pullover
3 : Dress
4 : Coat
5 : Sandal
6 : Shirt
7 : Sneaker
8 : Bag
9 : Ankle Boot
```

#02 Pytorch 라이브러리

#1 Pytorch 라이브러리 호출

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 import torch
5 import torch.nn as nn
6 from torch.autograd import Variable
7 import torch.nn.functional as F
8
9 import torchvision
10 import torchvision.transforms as transforms
11 from torch.utils.data import Dataset, DataLoader
```

#2 라이브러리 및 패키지 설명

Numpy: 수치 계산을 위한 라이브러리

matplotlib: 데이터 시각을 위한 라이브러리

torch: 텐서를 다룸

torch.nn: 신경망을 구성하는데 필요한 계층, 손실함수가 들어있음

torch.autograd: 자동 미분 패키지. 역전파 자동 계산

torch.nn.functional: 자주 사용하는 함수 제공 (함수, 풀링 연산 함수 등)

torchvision: 이미지 데이터를 다루기 위한 패키지

transforms: 데이터 전처리를 위해 사용하는 라이브러리

- Dataset

- 데이터와 레이블을 저장하는 클래스
- 인덱싱 -> 특정 데이터 불러오기

- DataLoader

- Dataset에 담긴 데이터를 모델에 전달
- Dataset을 배치 크기만큼 나누어 처리
- 기능: 미니배치, 셔플, 병렬처리

#3 클래스 설명

#03 CPU/GPU 장치 선택

#1 CPU/GPU 장치 확인 코드

```
1 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

#2 하나의 CPU를 사용할 때

```
1 # 하나의 CPU
2 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
3 model = Net()
4 model.to(device)
```

#3 다수의 GPU를 사용할 때

```
6 # 다수의 GPU
7 device = torch.device("cuda" if torch.is_available() else "cpu")
8 model = Net()
9 if torch.cuda.device_count() > 1:
10     model = nn.DataParallel(net) # batch size가 알려져 각 GPU로 분배
11 model.to(device)
```


#04 데이터셋 로드, DataLoader에 전달

#1 데이터셋 로드

①

②

③

```
1 train_dataset =  
2 torchvision.datasets.FashionMNIST("../chap05/data", download=True, transform=  
3                                     transforms.Compose([transforms.ToTensor()])))  
4 test_dataset =  
5 torchvision.datasets.FashionMNIST("../chap05/data", download=True, train=False, transform=  
6                                     transforms.Compose([transforms.ToTensor()])))
```

- ① FashionMNIST 내려받을 위치 지정
- ② True => 지정된 위치에 데이터셋 있는지 확인하고 다운로드
- ③ 이미지를 텐서(0~1)로 변경

#2 DataLoader에 전달

```
1 from IPython.testing import test  
2 train_loader = torch.utils.data.DataLoader(train①dataset, batch_size=100)  
3 test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=100)
```

- ① 데이터 로드할 데이터셋 지정
- ② 데이터를 묶는 단위
e.g) batch_size = 100
100개 단위로 데이터 묶어서 로드

torch.utils.data.DataLoader()

- 원하는 크기의 배치 단위로 데이터셋 로드 가능
- shuffle 가능

#05 이미지 전처리와 텐서 변환의 이유

```
1 labels_map = {0 : 'T-Shirt', 1 : 'Trouser', 2 : 'Pullover', 3 : 'Dress', 4 : 'Coat', 5 : 'Sandal', 6 : 'Shirt',
2               7 : 'Sneaker', 8 : 'Bag', 9 : 'Ankle Boot'}
3
4 fig = plt.figure(figsize=(8, 8))
5 columns = 4
6 rows = 5
7 for i in range(1, columns*rows+1):
8     img_xy = np.random.randint(len(train_dataset)) ①
9     img = train_dataset[img_xy][0][0,:,:]          ②
10    fig.add_subplot(rows, columns, i)
11    plt.title(labels_map[train_dataset[img_xy][1]])
12    plt.axis('off')
13    plt.imshow(img, cmap='gray')
14 plt.show()
```

- ① np.random: 무작위로 데이터 생성
np.random.randint: 이산형 분포를 갖는 데이터에서 무작위 표본 추출
np.random.randint(len(train_dataset)): 0~ 길이에서 랜덤 숫자 생성
- ② train_dataset 이용해 3차원 배열 생성
해당 인덱스에 해당하는 요소 값을 가져옴



#06 DNN 모델 정의

```
1 class FashionDNN(nn.Module): ①
2     def __init__(self): ②
3         super(FashionDNN,self).__init__() ③
4         self.fc1 = nn.Linear(in_features=784,out_features=256) ④
5         self.drop = nn.Dropout2d(0.25) ⑤
6         self.fc2 = nn.Linear(in_features=256,out_features=128)
7         self.fc3 = nn.Linear(in_features=128,out_features=10)
8
9     def forward(self,input_data): ⑥
10        out = input_data.view(-1, 784) ⑦
11        out = F.relu(self.fc1(out)) ⑧
12        out = self.drop(out)
13        out = F.relu(self.fc2(out))
14        out = self.fc3(out)
15        return out
```

- ① torch.nn.Module 상속받아 클래스 형태의 모델로 정의
- ② 객체가 갖는 속성값을 초기화. 객체 생성 시 자동 호출
- ③ 부모 클래스 FashionDNN 상속받기
- ④
 - nn: 딥러닝 모델 구성에 필요한 모듈 패키지
 - Linear: 단순 선형 회귀 모델
 - 파라미터
 - in_features: 입력의 크기
 - out_features: 출력의 크기
- ⑤ torch.nn.Dropout(p)
p만큼의 비율로 텐서의 값이 0됨.
0되지 않는 값들은 기존값 * 1/(1-p)해서 커짐
- ⑥ forward() 함수
 - 순전파 학습 진행
 - 함수 이름은 반드시 "forward"로 지정
- ⑦ view
 - 텐서의 크기 변경 (numPy의 reshape 역할)
 - input_data
- ⑧ 활성화 함수 지정

#06 DNN 모델 정의

활성화 함수 지정 방법

```
1 # nn 사용하는 코드
2 import torch
3 import torch.nn as nn
4
5 inputs = torch.Tensor(64,3,244,244)
6 conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1)
7 outputs = conv(inputs)
8 layer = nn.Conv2d(1,1,3)
```

```
1 # nn.functional 사용하는 코드
2 import torch
3 import torch.nn.functional as F
4 inputs = torch.randn(64,3,244,244)
5 weight = torch.randn(64,3,3,3)
6 bias = torch.randn(64)
7 outputs = F.conv2d(inputs,weight,bias, padding=1)
```

1 nn 사용

- nn.Conv2d: 클래스
- nn.Module: 클래스를 상속받아 사용
- 호출 방법: 1. 하이퍼파라미터 전달 2. 함수호출
- nn.Sequential 내에 위치
- 파라미터 새로 정의할 필요 없음

2 nn.functional 사용

- nn.functional.conv2d: 함수. def function (input)으로 정의됨
- 호출 방법: 함수 호출 시 하이퍼파라미터, 데이터 전달
- nn.Sequential에 위치할 수 없음
- 가중치를 수동으로 전달해야할 때마다 자체 가중치를 정의함

#07 순전파 및 역전파 과정

1 심층 신경망에서 필요한 파라미터 정의

```
1 learning_rate = 0.001;
2 model = FashionDNN();
3 model.to(device)
4
5 criterion = nn.CrossEntropyLoss();
6 optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate);
7 print(model)
```

```
... FashionDNN(
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (drop): Dropout2d(p=0.25, inplace=False)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (fc3): Linear(in_features=128, out_features=10, bias=True)
)
```

경사하강법: Adam
학습률(lr): 0.001

#07 순전파 및 역전파 과정

#2 심층 신경망을 이용한 모델 학습

```
1 num_epochs = 5
2 count = 0
3 loss_list = []
4 iteration_list = []
5 accuracy_list = []
6
7 predictions_list = []
8 labels_list = []
9
```

loss_list[]: 손실 저장할 빈 리스트

iteration_list[]: 반복횟수 저장할 빈 리스트

accuracy_list[]: 정확도를 저장할 빈 리스트

prediction_list[]: 모델의 예측값을 저장할 빈 리스트

labels_list[]: 레이블을 저장할 빈 리스트

#07 순전파 및 역전파 과정

#2 심층 신경망을 이용한 모델 학습

```
10 for epoch in range(num_epochs):
11     for images, labels in train_loader: ①
12         images, labels = images.to(device), labels.to(device) ②
13
14         train = Variable(images.view(100, 1, 28, 28)) ③
15         labels = Variable(labels)
16
17         outputs = model(train)
18         loss = criterion(outputs, labels)
19         optimizer.zero_grad()
20         loss.backward()
21         optimizer.step()
22         count += 1
```

- ① 레코드에서 요소 두개 추출
* for in 구문
for x, y in train:
의미: 레코드에서 요소 두개 추출
- ② 모델과 데이터가 동일한 device에 위치함
- ③ Variable 사용
-> Autograd: Variable를 이용해
역전파를 위한 미분값 자동 계산

#07 순전파 및 역전파 과정

#2 심층 신경망을 이용한 모델 학습

```
23
24     if not (count % 50):
25         total = 0
26         correct = 0
27         for images, labels in test_loader:
28             images, labels = images.to(device), labels.to(device)
29             labels_list.append(labels)
30             test = Variable(images.view(100, 1, 28, 28))
31             outputs = model(test)
32             predictions = torch.max(outputs, 1)[1].to(device)
33             predictions_list.append(predictions)
34             correct += (predictions == labels).sum()
35             total += len(labels)
36
37         accuracy = correct * 100 / total ④
38         loss_list.append(loss.data) ⑤
39         iteration_list.append(count)
40         accuracy_list.append(accuracy)
41
42     if not (count % 500):
43         print("Iteration: {}, Loss: {}, Accuracy: {}".format(count, loss.data, accuracy))
```

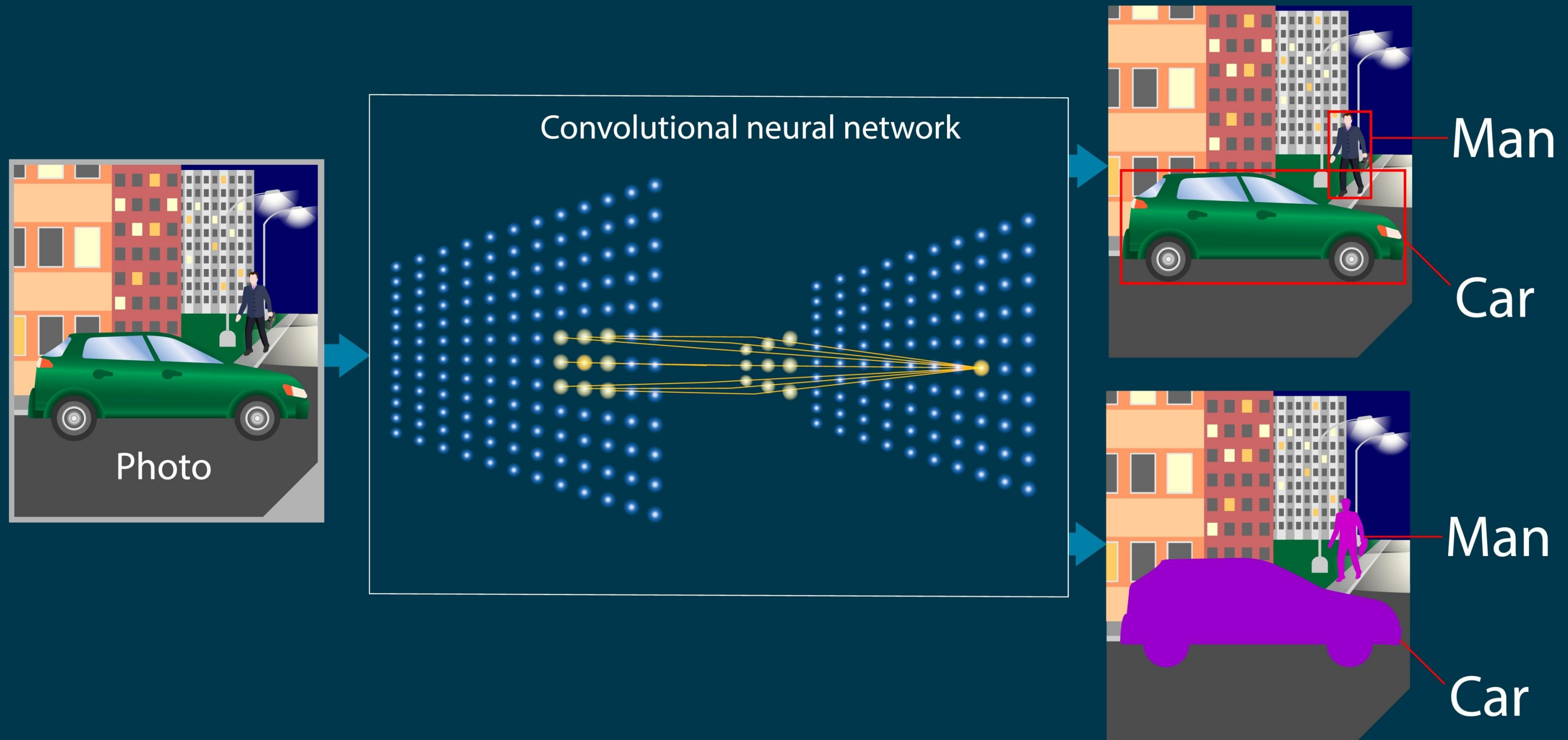
```
... /usr/local/lib/python3.12/dist-packages/torch/nn/modules/dropout.py:176: UserWarning: dropout2d: Recei
    return F.dropout2d(input, self.p, self.training, self.inplace)
Iteration: 500, Loss: 0.549751877784729, Accuracy: 83.12000274658203%
Iteration: 1000, Loss: 0.4469604790210724, Accuracy: 84.58000183105469%
Iteration: 1500, Loss: 0.3575650155544281, Accuracy: 83.37999725341797%
Iteration: 2000, Loss: 0.3772505223751068, Accuracy: 85.80000305175781%
Iteration: 2500, Loss: 0.243845596909523, Accuracy: 86.31999969482422%
Iteration: 3000, Loss: 0.2834920287132263, Accuracy: 86.26000213623047%
```

- ④ 전체 예측에 대한 정확한 예측의 비율
- ⑤ append 메소드를 이용해 데이터 추가

실행 결과: 정확도 86%

#07 합성곱 신경망(CNN)

Object detection and Instance segmentation

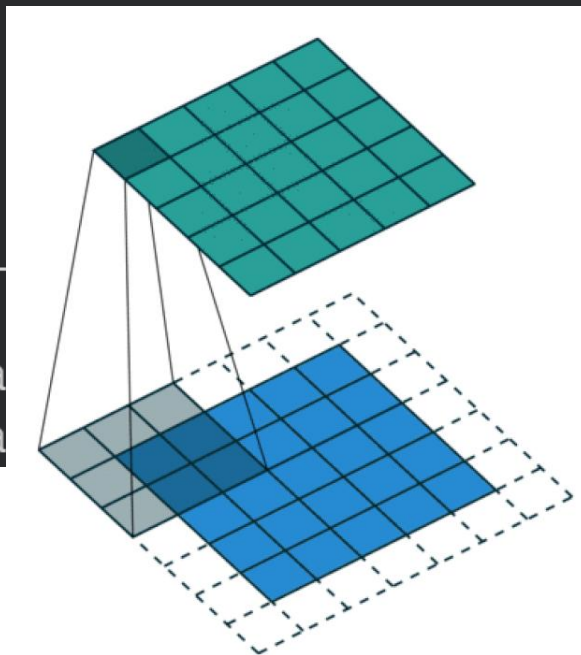


#07 합성곱 신경망(CNN)

#2 합성곱 네트워크 생성

$$Y_k(i, j) = \sum_{m=0}^2 \sum_{n=0}^2 X_{pad}(i+m, j+n) \cdot W_k(m, n) + b_k$$

```
1 class FashionCNN(nn.Module):
2     def __init__(self):
3         super(FashionCNN, self).__init__()
4         self.layer1 = nn.Sequential(
5             nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
6             nn.BatchNorm2d(32),
7             nn.ReLU(),
8             nn.MaxPool2d(kernel_size=2, stride=2)
9         )
10        self.layer2 = nn.Sequential(
11            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
12            nn.BatchNorm2d(64),
13            nn.ReLU(),
14            nn.MaxPool2d(2)
15        )
16        self.fc1 = nn.Linear(in_features=64*6*6, out_
17        self.drop = nn.Dropout2d(0.25)
18        self.fc2 = nn.Linear(in_features=600, out_fea
19        self.fc3 = nn.Linear(in_features=120, out_fea
```



$$\text{출력 크기} = (W - F + 2P) / S + 1$$

- nn.Sequential은 여러 개의 계층을 하나의 컨테이너에 구현하는 방법
- in_channels=1: 흑백 이미지이므로 입력 채널이 1입니다 (컬러면 3).
- out_channels=32: 32개의 다양한 필터(특징 맵)를 만들어냅니다.
- kernel_size=3: 3x3 크기의 필터를 사용해 이미지를 훑습니다.
- padding=1: 모서리 부분의 데이터 손실을 막기 위해 이미지 곁에 0을 한 줄 두릅니다. (28x28 이미지가 그대로 28x28을 유지합니다.)
- nn.MaxPool2d(kernel_size=2, stride=2) : 풀링 계층입니다. 2x2 영역에서 가장 큰 값만 남겨 이미지 크기를 절반(14x14)으로 줄입니다. 연산량을 줄이고 중요한 특징만 남깁니다.

#07 합성곱 신경망(CNN)

#2 합성곱 네트워크 생성

```
1 class FashionCNN(nn.Module):
2     def __init__(self):
3         super(FashionCNN, self).__init__()
4         self.layer1 = nn.Sequential(
5             nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
6             nn.BatchNorm2d(32),
7             nn.ReLU(),
8             nn.MaxPool2d(kernel_size=2, stride=2)
9         )
10        self.layer2 = nn.Sequential(
11            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
12            nn.BatchNorm2d(64),
13            nn.ReLU(),
14            nn.MaxPool2d(2)
15        )
16        self.fc1 = nn.Linear(in_features=64*6*6, out_features=600)
17        self.drop = nn.Dropout2d(0.25)
18        self.fc2 = nn.Linear(in_features=600, out_features=120)
19        self.fc3 = nn.Linear(in_features=120, out_features=10)
```

- self.fc1 = nn.Linear(in_features=64*6*6, out_features=600)
- 입력 크기: 64채널 * 6가로 * 6세로 = 2304. 평면으로 펼쳐진 2304개의 데이터를 600개로 압축합니다.
- self.drop = nn.Dropout2d(0.25)
- 과적합(Overfitting, 모델이 훈련 데이터에만 너무 익숙해지는 현상)을 막기 위해 뉴런의 25%를 무작위로 끄면서 학습합니다.
- self.fc2 & self.fc3
- 600개 -> 120개 -> 10개로 데이터를 서서히 줄여갑니다. 마지막 10은 FashionMNIST의 10가지 옷 종류(티셔츠, 바지, 가방 등)를 의미합니다.

out.view(out.size(0), -1) : 4차원 -> 2차원

#07 합성곱 신경망(CNN)

#2 합성곱 네트워크를 위한 파라미터 정의

```
1 learning_rate = 0.001;
2 model = FashionCNN();
3 model.to(device)
4
5 criterion = nn.CrossEntropyLoss();
6 optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate);
7 print(model)
```

```
FashionCNN(
  (layer1): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (layer2): Sequential(
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (fc1): Linear(in_features=2304, out_features=600, bias=True)
  (drop): Dropout2d(p=0.25, inplace=False)
  (fc2): Linear(in_features=600, out_features=120, bias=True)
  (fc3): Linear(in_features=120, out_features=10, bias=True)
)
```

THANK YOU

